

Persistencia en Cascada con JPA y Jakarta EE



Persistencia en Cascada con JPA en Jakarta EE

En esta guía, aprenderemos a utilizar **persistencia en cascada** en **JPA** para gestionar relaciones entre entidades de manera eficiente. Este concepto permite que cuando persistimos, eliminamos o actualizamos una entidad principal, JPA automáticamente realice la misma operación en las entidades relacionadas.

La persistencia en cascada es útil cuando trabajamos con entidades que tienen relaciones, como **uno a muchos (OneToMany)** o **muchos a uno (ManyToOne)**. En este caso, trabajaremos con las clases `Persona` y `Usuario`, donde una persona puede tener múltiples usuarios asociados.

1. Conceptos Básicos

Antes de comenzar con el código, revisemos algunos conceptos clave:

1.1 ¿Qué es la persistencia en cascada?

La persistencia en cascada permite que las operaciones realizadas en una entidad principal se propaguen automáticamente a las entidades relacionadas. Se usa el atributo `cascade` en las anotaciones de relaciones en JPA, como `@OneToMany` y `@ManyToOne`.

1.2 Tipos de Cascade en JPA

JPA proporciona varios tipos de **CascadeType**, entre ellos:

- **CascadeType.PERSIST**: Propaga la operación de persistencia.
- **CascadeType.MERGE**: Propaga la operación de actualización.
- **CascadeType.REMOVE**: Propaga la operación de eliminación.
- **CascadeType.REFRESH**: Recarga la entidad desde la base de datos.
- **CascadeType.ALL**: Aplica todas las operaciones anteriores.

En esta guía, usaremos **CascadeType.ALL**, lo que significa que cualquier operación realizada en `Persona` también afectará a `Usuario`.

1.3 Sintaxis Básica

Ejemplo de configuración de persistencia en cascada en una relación **uno a muchos**:

```
@OneToMany(mappedBy = "persona", cascade = CascadeType.ALL)
private List<Usuario> usuarioList;
```

Esto indica que si persistimos un objeto de tipo `Persona`, automáticamente se guardarán los `Usuario` asociados.

2. Implementación Paso a Paso

A continuación, implementaremos la persistencia en cascada en JPA.

Paso 1: Configurar la Entidad `Persona`

La clase `Persona` representa a una persona en el sistema. Está relacionada con `Usuario` mediante una relación **uno a muchos**.

```
package sga.domain;

import jakarta.persistence.*;
import jakarta.validation.constraints.Size;
import java.io.Serializable;
import java.util.List;

@Entity
@NamedQueries({
    @NamedQuery(name = "Persona.findAll", query = "SELECT p FROM Persona p"),
    @NamedQuery(name = "Persona.findByIdPersona", query = "SELECT p FROM Persona p WHERE p.idPersona = :idPersona"),
    @NamedQuery(name = "Persona.findByName", query = "SELECT p FROM Persona p WHERE p.nombre = :nombre"),
    @NamedQuery(name = "Persona.findByApellido", query = "SELECT p FROM Persona p WHERE p.apellido = :apellido"),
})
```

```

@NamedQuery(name = "Persona.findByEmail", query = "SELECT p FROM Persona p WHERE p.email = :email"),
@NamedQuery(name = "Persona.findByTelefono", query = "SELECT p FROM Persona p WHERE p.telefono = :telefono"))}

public class Persona implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "id_persona")
    private Integer idPersona;
    @Size(max = 45)
    private String nombre;
    @Size(max = 45)
    private String apellido;
    // @Pattern(regexp="[a-z0-9!#$%&'*/+=?^_`{|}~-]+(?:\\.\\.[a-z0-9!#$%&'*/+=?^_`{|}~-]+)*@(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?)?\\.\\.[a-z0-9](?:[a-z0-9-]*[a-z0-9])?", message="Invalid email")//if the field contains email address consider using
    this annotation to enforce field validation
    @Size(max = 45)
    private String email;
    @Size(max = 45)
    private String telefono;
    @OneToMany(mappedBy = "persona", cascade = CascadeType.ALL)
    private List<Usuario> usuarioList;

    public Persona() {
    }

    public Persona(String nombre, String apellido, String email, String telefono) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.telefono = telefono;
    }

    public Persona(Integer idPersona) {
        this.idPersona = idPersona;
    }

    public Integer getIdPersona() {
        return idPersona;
    }

    public void setIdPersona(Integer idPersona) {
        this.idPersona = idPersona;
    }
}

```

```
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
    this.telefono = telefono;
}

public List<Usuario> getUsuarioList() {
    return usuarioList;
}

public void setUsuarioList(List<Usuario> usuarioList) {
    this.usuarioList = usuarioList;
}

@Override
public int hashCode() {
    int hash = 0;
    hash += (idPersona != null ? idPersona.hashCode() : 0);
}
```

```

        return hash;
    }

    @Override
    public boolean equals(Object object) {
        // TODO: Warning - this method won't work in the case the id fields are not set
        if (!(object instanceof Persona)) {
            return false;
        }
        Persona other = (Persona) object;
        if ((this.idPersona == null && other.idPersona != null) || (this.idPersona != null &&
!this.idPersona.equals(other.idPersona))) {
            return false;
        }
        return true;
    }

    @Override
    public String toString() {
        return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", apellido=" + apellido + ", email=" +
email + ", telefono=" + telefono + '}';
    }
}

```

◆ Explicación:

- Se usa `@Entity` para indicar que `Persona` es una entidad JPA.
- `@OneToMany(mappedBy = "persona", cascade = CascadeType.ALL)` indica que la relación con `Usuario` es **uno a muchos** y que se usará **persistencia en cascada**.

Paso 2: Configurar la Entidad `Usuario`

La entidad `Usuario` representa un usuario en el sistema y está relacionada con `Persona`.

```

package sga.domain;

import jakarta.persistence.*;
import jakarta.validation.constraints.Size;
import java.io.Serializable;

@Entity
@NamedQueries({

```

```
@NamedQuery(name = "Usuario.findAll", query = "SELECT u FROM Usuario u"),
@NamedQuery(name = "Usuario.findByIdUsuario", query = "SELECT u FROM Usuario u WHERE u.idUsuario = :idUsuario"),
@NamedQuery(name = "Usuario.findByUsername", query = "SELECT u FROM Usuario u WHERE u.username = :username"),
@NamedQuery(name = "Usuario.findByPassword", query = "SELECT u FROM Usuario u WHERE u.password = :password"))}

public class Usuario implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "id_usuario")
    private Integer idUsuario;

    @Size(max = 45)
    private String username;
    @Size(max = 45)
    private String password;

    @JoinColumn(name = "id_persona", referencedColumnName = "id_persona")
    @ManyToOne(cascade = CascadeType.ALL)
    private Persona persona;

    public Usuario() {
    }

    public Usuario(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public Usuario(Integer idUsuario) {
        this.idUsuario = idUsuario;
    }

    public Usuario(String username, String password, Persona persona) {
        this.username = username;
        this.password = password;
        this.persona = persona;
    }

    public Integer getIdUsuario() {
        return idUsuario;
    }

    public void setIdUsuario(Integer idUsuario) {
```

```
        this.idUsuario = idUsuario;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public Persona getPersona() {
        return persona;
    }

    public void setPersona(Persona persona) {
        this.persona = persona;
    }

    @Override
    public int hashCode() {
        int hash = 0;
        hash += (idUsuario != null ? idUsuario.hashCode() : 0);
        return hash;
    }

    @Override
    public boolean equals(Object object) {
        // TODO: Warning - this method won't work in the case the id fields are not set
        if (!(object instanceof Usuario)) {
            return false;
        }
        Usuario other = (Usuario) object;
        if ((this.idUsuario == null && other.idUsuario != null) || (this.idUsuario != null &&
!this.idUsuario.equals(other.idUsuario))) {
            return false;
        }
    }
```

```

        return true;
    }

    @Override
    public String toString() {
        return "Usuario{" + "idUsuario=" + idUsuario + ", username=" + username + ", password=" + password + ",
persona=" + persona + '}';
    }
}

```

◆ Explicación:

- Se usa `@Entity` para definir `Usuario` como una entidad JPA.
- `@ManyToOne` indica que cada usuario pertenece a una `Persona`.
- `@JoinColumn(name = "id_persona")` especifica la clave foránea que enlaza con `Persona`.

Paso 3: Implementar `PersistenciaCascadaJPA.java`

Ahora probaremos la persistencia en cascada creando objetos de `Persona` y `Usuario`.

```

package sga.cliente.cascade;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.EntityTransaction;
import jakarta.persistence.Persistence;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import sga.domain.Persona;
import sga.domain.Usuario;

public class PersistenciaCascadaJPA {

    private static final Logger log = LoggerFactory.getLogger(PersistenciaCascadaJPA.class);

    public static void main(String[] args) {

        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
        try (EntityManager em = emf.createEntityManager()) {
            EntityTransaction tx = em.getTransaction();
            tx.begin();

            // 1. Crear objeto Persona

```



```

Persona persona1 = new Persona("Hugo", "Hernandez", "hhernandez@mail.com", "55778822");

// 2. Crear objeto Usuario relacionado con Persona
Usuario usuario1 = new Usuario("hhernandez", "123", persona1);

// 3. Persistir solo el Usuario (en automatico inserta la persona tambien)
em.persist(usuario1);

// 4. Confirmar transacción
tx.commit();

log.info("Persona guardada: " + persona1);
log.info("Usuario guardado: " + usuario1);
}
}
}

```

◆ Explicación:

1. Se crea un objeto **Persona** con nombre, apellido, email y teléfono.
2. Se crea un objeto **Usuario** relacionado con **Persona**.
3. Se persiste solo el **Usuario**, lo que debido a la **persistencia en cascada** también guarda **Persona** automáticamente.
4. Se realiza el **commit** para aplicar los cambios en la base de datos.

Paso 4: Resultado Esperado

Al ejecutar el código, se espera que en la base de datos se inserten:

1. Un registro en la tabla **Persona**.
2. Un registro en la tabla **Usuario**, enlazado con **Persona**.

En la consola se imprimirá:

```
[EL Fine]: sql: ClientSession(1415439780)--Connection(703555670)--INSERT INTO PERSONA (APELLIDO, EMAIL, NOMBRE, TELEFONO) VALUES (?, ?, ?, ?)
```

```
bind => [Hernandez, hhernandez@mail.com, Hugo, 55778822]
```

```
[EL Fine]: sql: ClientSession(1415439780)--Connection(703555670)--INSERT INTO USUARIO (PASSWORD, USERNAME, id_persona) VALUES (?, ?, ?)
```

```
bind => [123, hhernandez, 5]
```

```
[main] INFO sga.cliente.cascade.PersistenciaCascadaJPA - Persona guardada: Persona{idPersona=5, nombre=Hugo, apellido=Hernandez, email=hhernandez@mail.com, telefono=55778822}
```

```
[main] INFO sga.cliente.cascade.PersistenciaCascadaJPA - Usuario guardado: Usuario{idUsuario=4, username=hhernandez, password=123, persona=Persona{idPersona=5, nombre=Hugo, apellido=Hernandez, email=hhernandez@mail.com, telefono=55778822}}
```

Esto confirma que la **persistencia en cascada** funciona correctamente.

Conclusión

✓ Hemos aprendido a utilizar la **persistencia en cascada con JPA** para manejar relaciones **uno a muchos** sin necesidad de persistir manualmente las entidades dependientes.



¡Ahora puedes aplicar este concepto en tus propios proyectos!

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)